Split tRNA prediction pseudo code

Phase I: genome scan and structural filters · Phase II: validation of predicted candidates ·

Phase I — Genome scan and structural filters

```
INPUT   genome G; length bounds Lmin, Lmax; number of randomisations NR
OUTPUT  two candidate sets: CAND (from real genome), DECOY (from shuffled)

STEP 1: PREPROCESS GENOME SEQUENCE AND BUILD SHUFFLED GENOMES
  read G; strip FASTA header, whitespace, digits, non-ACGT symbols
  uppercase; convert T to U for scoring
  GENOMES <- { G }                                    // the real genome
  for run <- 1 to NR:
      GENOMES <- GENOMES + { randomise(G) }           // preserve length and base
                                                      // composition; dinucleotide-
                                                      // preserving shuffle preferred
  // every genome below passes through the IDENTICAL steps 2-10

STEP 2: ANALYSE BOTH STRANDS
  for each genome g in GENOMES:
    for STRAND in {FORWARD, REVERSE}:
        seq <- g if FORWARD else reverse complement(g)

STEP 3: SEARCH POSSIBLE 5' HALVES
        for each position n in seq:
            segment <- seq[n .. n+40]
            if check first(segment):
                record (first half start, first half length)

STEP 4: SEARCH POSSIBLE 3' HALVES
        for each position n1 in seq:
            segment <- required 3' region from n1
            if check 2nd(segment):
                record second half position

STEP 5: ASSEMBLE CANDIDATE SPLIT tRNA
        for each 5' half F, each 3' half S, each L in [Lmin, Lmax]:
            Candidate <- first part(F) + second part(S)
            if length(Candidate) != L: continue

STEP 6: INITIALISE STRUCTURE
            x[1..72] <- ALIGN SPRINZL(Candidate)    // ADDED: map onto the 72
                                                    // standard positions;
                                                    // unfilled slots = GAP
            if alignment fails: reject
            initialise cloverleaf matrix on x
            acceptor_arm   <- (1,72)(2,71)(3,70)(4,69)(5,68)(6,67)(7,66)
            D_arm          <- (10,25)(11,24)(12,23)(13,22)
            anticodon arm  <- (27,43)(28,42)(29,41)(30,40)(31,39)
            T arm          <- (49,65)(50,64)(51,63)(52,62)(53,61)
            tertiary pairs <- 8-14, 15-48, 18-55, 19-56, 26-44, 32-38, 54-58
            // the mapping x is stored with the candidate: every positional
            // claim downstream depends on it and must be auditable

STEP 7: TEST TOTAL STEM
            a pair is paired if A-U, U-A, G-C, C-G, G-U or U-G
            a pair with a GAP on either side is NOT SCORED (not a mismatch)
            for arm in {acceptor, D, anticodon, T}:
                if mismatches(arm) > arm threshold: REJECT candidate
            paired <- pairs paired over all 21 canonical pairs
            scored <- pairs with both sides present
            P pair <- paired / scored        // recorded, never used to decide

STEP 8: TEST CONSERVED POSITIONS
            INVARIANT <- {8:U, 14:A, 21:A, 33:U, 48:C, 53:G,
                          54:U, 55:U, 56:C, 57:G, 58:A, 61:C}
            TLOOP     <- {54:U, 55:U, 56:C, 57:G, 58:A}
            I      <- matches over INVARIANT / INVARIANT positions present
            Theta  <- matches over TLOOP / TLOOP positions present
            Levitt <- (x[15] = G and x[48] = C)
            anticodon <- x[34] x[35] x[36]           // ADDED: extract here
            if anticodon != declared anticodon: flag mismatch

STEP 9: TEST TERTIARY INTERACTIONS
            count deviations over tertiary_pairs
            if stem deviation + tertiary deviation > 3: REJECT candidate

STEP 10: SPLIT-JUNCTION VALIDATION
            extract genomic context flanking both halves
            test BHB-like / bulge-helix compatibility with bh loop()
            if bh loop() does not exist: REJECT candidate
            S_split <- split-context score
```

```
STEP 11: TWO SETS OF OUTPUT CANDIDATES
            GC      <- (G+C) / non-GAP positions     // ADDED: needed in II
            GC stem <- GC over the 42 stem positions
            emit (x, anticodon, P pair, I, Theta, Levitt, GC, GC stem, S split)
            if g is the real genome: add to CAND
            else:                    add to DECOY    // the method's own
                                                     // false positives
  fp yield <- size(DECOY) / NR
  pass CAND and DECOY to PHASE II
```

Phase II — Validation of the predicted candidates

```
STEP 12: INPUTS
  REF    reference set, 200 known tRNAs on Sprinzl positions 1-72
  VAL1   known tRNAs of the same species (mature), held out of REF
  VAL2   further known tRNAs from the reference database, held out of REF
  DECOY  candidates from the shuffled genomes            (from PHASE I)
  CAND   predicted candidates to validate                (from PHASE I)
  alpha <- 1                                             // Laplace smoothing

--- Build the reference probability model -------------------------------
STEP 13: READ THE REFERENCE SET BY POSITION LABEL
  for each record r in REF, each position i in 1..72:
      REF[r][i] <- normalise(cell)    // T->U; 'Absent', '-', blank -> GAP
  assert the 72 columns are those LABELLED 1..72
  // taking the first 72 data columns instead shifts every position from 18
  // onward and silently drops 69-72
  assert no position has N i = 0

STEP 14: FREQUENCY AND EMISSION MATRIX
  for i <- 1 to 72:
      n_i(b) <- count of records with REF[r][i] = b,  for b in {A,C,G,U}
      N_i    <- n_i(A)+n_i(C)+n_i(G)+n_i(U)            // GAPs excluded
      e_i(b) <- ( n_i(b) + alpha ) / ( N_i + 4*alpha )
      assert sum over b of e_i(b) = 1

STEP 15: TRANSITION MATRICES AND BACKGROUNDS
  for i <- 1 to 71:
      c_i(b1,b2) <- count with REF[r][i]=b1 and REF[r][i+1]=b2
      Cx_i(b1)   <- sum over b2 of c_i(b1,b2)
      t_i(b2|b1) <- ( c_i(b1,b2) + alpha ) / ( Cx_i(b1) + 4*alpha )
      assert sum over b2 of t_i(b2|b1) = 1
  q(b)       <- sum_i n_i(b) / sum {i,b} n_i(b)
  q_d(b2|b1) <- ( sum_i c_i(b1,b2) + alpha )
                / ( sum {i,b2} c_i(b1,b2) + 4*alpha )

--- Calibrate on known sets, build the two nulls ------------------------
STEP 16: SCORE THE REFERENCE SET, LEAVE ONE OUT
  for each r in REF:
      rebuild e_i, t_i with r's own counts subtracted
      S_r <- SCORE(r, leave one out = r)       // SCORE defined in STEP 20
  // without leave-one-out each record contributes to its own score

STEP 17: SCORE THE KNOWN VALIDATION SETS
  deduplicate VAL1, VAL2 by accession; assert VAL1 and REF do not overlap
  POS <- { SCORE(x) : x in VAL1 + VAL2 }
  mean_real, sd_real, min_real <- mean, sd, min of POS
  I_5     <- 5th percentile of I over POS
  Theta_5 <- 5th percentile of Theta over POS

STEP 18: NULL A - COMPOSITION-MATCHED PERMUTATION (easy null)
  NULL_A <- empty
  for each x in VAL1, repeat K times:
      slots <- positions where x[i] != GAP
      permute x's own bases across its own slots       // GAPs stay GAPs
      NULL_A <- NULL_A + { SCORE(permuted x) }
  // holds composition exactly; destroys only positional order

STEP 19: NULL B - RANDOMIZED GENOME (realistic null)
  NULL_B <- { SCORE(y) : y in DECOY }                  // from PHASE I
  // same predictor, same filters, same anticodon classes as CAND

STEP 19b: POWER CHECK ON NULL B
  p_min <- 1 / ( size(NULL_B) + 1 )
  if p_min > 0.05:
      WARN "null too small: no candidate can reach p < 0.05"
      WARN "at least 99 decoys needed for p < 0.01,
            i.e. ceil(99 / fp yield) randomised genomes"
  // RECOMMENDED: also build a GC-matched NULL_B, since GC is the feature
  // that separates CAND from DECOY (see STEP 23)

--- Probabilistic scoring -----------------------------------------------
STEP 20: FUNCTION SCORE(x, leave one out = none)
  E <- mean over { i : x[i] != GAP } of ln[ e_i(x[i]) / q(x[i]) ]
```

```
  T <- mean over { i : x[i], x[i+1] != GAP } of
                       ln[ t i(x[i+1]|x[i]) / q d(x[i+1]|x[i]) ]
  S <- ( E + T ) / 2
  return (E, T, S, n E, n T)
  // gaps skipped; transitions spanning a gap DROPPED, not bridged
  // means, not sums: otherwise a longer candidate scores higher for being
  // longer, and candidates with different gap counts are not comparable

--- Threshold calibration and the GC check -----------------------------
STEP 21: ROC AGAINST EACH NULL
  for NULL in {NULL_A, NULL_B}:
      AUC <- Mann-Whitney statistic of POS against NULL, tie-corrected
      null max <- max of NULL
      if null max < min real:
          tau <- ( null max + min real ) / 2
          // separation perfect: every cut in the band is equivalent, so the
          // midpoint is a stated choice, not an optimum - say so
      else:
          tau <- threshold maximising Youden J
  tau A <- cut from NULL A (easy)   tau B <- cut from NULL B (realistic; USE)

STEP 22: EMPIRICAL p FOR A CANDIDATE
  p A <- ( count{ s in NULL A : s >= S(x) } + 1 ) / ( size(NULL A) + 1 )
  p B <- ( count{ s in NULL B : s >= S(x) } + 1 ) / ( size(NULL B) + 1 )
  z   <- ( S(x) - mean real ) / sd real

STEP 23: GC-ADJUSTED SEPARATION CHECK  (on the SET, not per candidate)
  POOL  <- CAND + DECOY;   label <- 1 for CAND, 0 for DECOY
  AUC S  <- AUC of S over POOL;    AUC GC <- AUC of GC over POOL
  fit S = a + b*GC over POOL;  resid <- S - (a + b*GC)
  p resid <- Mann-Whitney p of resid, CAND vs DECOY
  if AUC GC >= AUC S: report "GC alone separates at least as well"
  if p resid > 0.05:  report "after removing GC the score adds no separation
                              - composition, not positional structure,
                              distinguishes the two sets"

--- Classification and output ------------------------------------------
STEP 24: ACCEPTANCE CRITERIA
  for each candidate x in CAND:
      A <- S(x) > tau B           B <- p B < 0.05
      C <- I >= I 5               D <- Theta >= Theta 5
      // P_pair is NOT a criterion: the search already discards anything
      // that cannot fold, so CAND and DECOY both fold well
      if A and C and D:                        STATUS <- SUPPORTED
      else if S(x) > max NULL B and (C or D):  STATUS <- BORDERLINE
      else:                                    STATUS <- NOT SUPPORTED
      if GC(x) > max GC over DECOY: FLAG "compositional confound"
      if p min > 0.05:              FLAG "p B at resolution floor"

STEP 25: TYPE ASSIGNMENT
      if candidate is contiguous: type <- canonical tRNA
      else:                      type <- split tRNA
      if anticodon in {UCA, CUA, UUA}: type <- split suppressor tRNA

STEP 26: OUTPUT RECORD
      report candidate id, strand, coordinates, both exon coordinates,
             genomic separation, mature sequence, Sprinzl mapping,
             anticodon, isotype, type,
             E, T, S with n E and n T,  z, p A, p B,
             tau A, tau B, max NULL B, min real,
             P_pair, I, Theta, Levitt, GC, GC_stem, S_split,
             STATUS and any FLAGs
END
```